

ECE 461 - HW03

1) Repetition coding: $\frac{\text{bit } 1}{\sqrt{E}}$ or $-\frac{\text{bit } 0}{\sqrt{E}}$ is sent n times

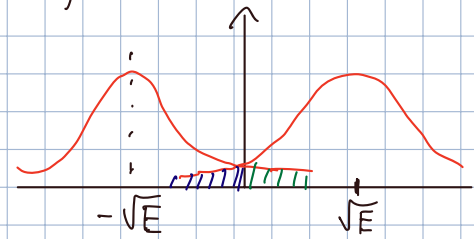
$$\sum_{m=1}^n \hat{y}[m] \underset{\text{bit } 1}{\overset{\text{bit } 0}{\gtrless}} 0 \Rightarrow \text{Decision rule}$$

(and choose randomly if $\sum_{m=1}^n \hat{y}[m] = 0$)

1.a) First calculate the error prob. over a single transmission

$$P(\text{single error} | \underbrace{x[m] = 0}_{\text{m}^{\text{th}} \text{ bit}}) = P(\underbrace{y[m] > 0}_{\text{m}^{\text{th}} \text{ voltage}} | \underbrace{v[m] = -\sqrt{E}}_{\text{m}^{\text{th}} \text{ voltage}}) = Q\left(\sqrt{\frac{E}{\sigma^2}}\right) = \rho$$

$$P(\text{single error} | x[m] = 1) = P(y[m] < 0 | v[m] = \sqrt{E}) = Q\left(\sqrt{\frac{E}{\sigma^2}}\right) = \rho$$



For odd $n = 2k+1$, $k \in \{0, 1, 2, \dots\}$

★ If bit sent is 0, transmitted bit. will be estimated incorrectly if there are least $\frac{n+1}{2} = k+1$ single erroneous transmissions.

$$P(\text{error} | x = 0) = \sum_{k=\frac{n+1}{2}}^n P[k \text{ single errors} | x = 0]$$

→ Since each transmission is indep. with failure probability ρ .
Total number of errors over n transmissions will be a binomial R.V.

$$P(k \text{ single errors} | x = 0) = \binom{n}{k} \rho^k (1-\rho)^{n-k}$$

$$P(\text{error} | x = 0) = \sum_{k=\frac{n+1}{2}}^n \binom{n}{k} \rho^k (1-\rho)^{n-k}$$

Similarly if the bit sent is 1, we need at least $\frac{n+1}{2}$ erroneous single transmissions again, with same p (symmetric voltages)

$$P(\text{error} | x=1) = \sum_{k=\frac{n+1}{2}}^n \binom{n}{k} p^k (1-p)^{n-k}$$

$$P_e = P(\text{error}) = \frac{1}{2} [P(\text{error} | x=0) + P(\text{error} | x=1)] = \boxed{\sum_{k=\frac{n+1}{2}}^n \binom{n}{k} p^k (1-p)^{n-k}}$$

For even $n=2k$

This transmitted symbol is decoded incorrectly if, there is at least $\frac{n}{2}+1$, erroneous single transmissions.

$$P(\text{error} | x=0) = \sum_{k=\frac{n}{2}+1}^n \binom{n}{k} p^k (1-p)^{n-k} + \frac{1}{2} \binom{n}{n/2} p^{n/2} (1-p)^{n/2}$$

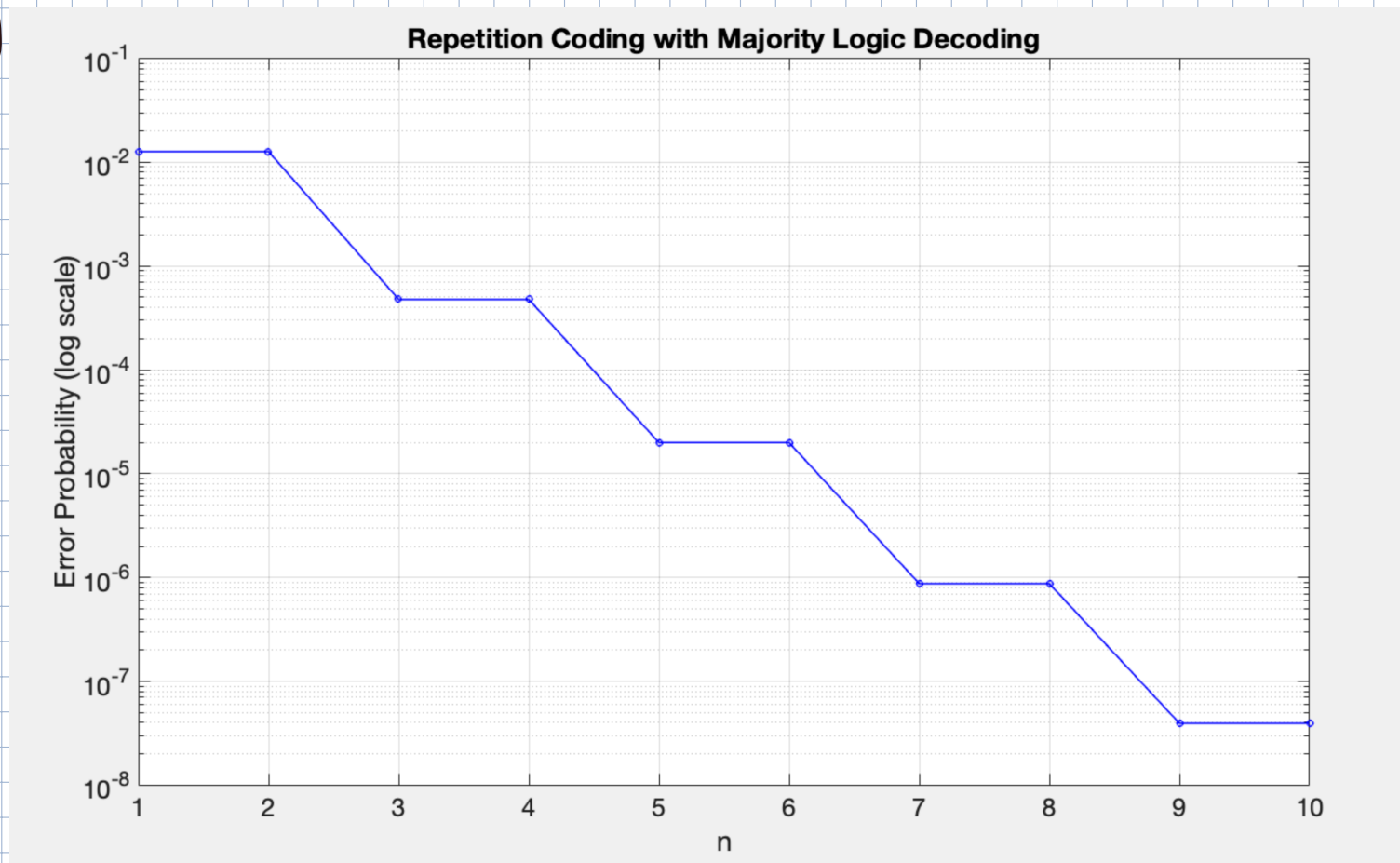
choosing
randomly
btwn 0 and 1

prob. that half of
transmitted symbols $y \in \mathcal{M} < 0$
and other half $y \in \mathcal{M} > 0$

$$P_e = P(\text{error}) = \frac{1}{2} P(\text{error} | x=0) + \frac{1}{2} \underbrace{P(\text{error} | x=1)}_{P(\text{error} | x=0)} =$$

$$= \boxed{\sum_{k=\frac{n}{2}+1}^n \binom{n}{k} p^k (1-p)^{n-k} + \frac{1}{2} \binom{n}{n/2} p^{n/2} (1-p)^{n/2}}$$

2.b)



2.c) We observe that subsequent error probs, with $n = 2m - 1$ for odd and $n = 2m$ for even, are the same.

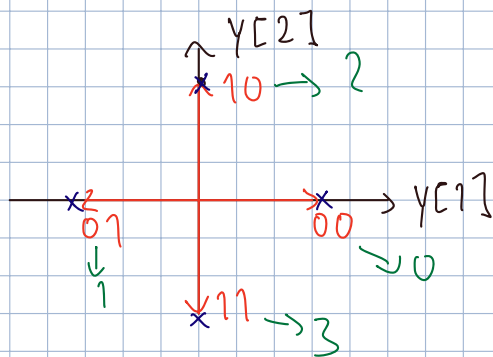
* We see that log error prob. decreases linearly, (except constant regions), This means error prob. decays exponentially.

In ML decoding for repetition coding:

$$P_e = \frac{2(N-1)}{N} Q\left(\frac{\sqrt{n \cdot \text{SNR}}}{N-1}\right) \sim e^{-\frac{1}{2} \frac{n \text{SNR}}{N-1}} \sim e^{-n}$$

→ Just as in ML decoding error decreases exponentially. So quantizing received voltage to $\{-1, 1\}$ doesn't affect the behaviour of P_e .

2) Let us start with $B=2$



→ The last bit determines whether symbol i is even or odd.

$$V_0 = (\sqrt{BE_b}, 0)$$

$$V_2 = (0, \sqrt{BE_b})$$

$$V_1 = (-\sqrt{BE_b}, 0)$$

$$V_3 = (0, -\sqrt{BE_b})$$

In general for bit seq. $(b_1, b_2, \dots, b_{2^{B-1}})$, we first convert it into decimal number representing the position of the nonzero voltage. Then we assign its parity based on b_{2^B} or whether i is odd or even

$$\vec{V}_i = (0, 0, \dots, (-1)^i \sqrt{BE_b}, \dots, 0, 0) \quad \text{where } i = 0, 1, \dots, 2^B - 1$$

$\uparrow$
 $\lfloor \frac{i}{2} \rfloor$ position

→ Intuitively we should first look at which received voltage has the largest absolute value, then we should decide on the last bit by looking at whether the voltage is positive or negative

↳ This is equivalent to applying ML rule in $B-1$ dimensions first and on the last dimension after

↳ Hence, we can combine all into a ML rule for voltages living in an $B-1$ dimensional space.

$$\begin{aligned} \hat{y}_i &= \arg \min_i \|\vec{a} - \vec{V}_i\|^2 = \arg \min_i \sum_{m=1}^{2^{B-1}} \underbrace{a^2(m)}_{\text{constant w.r.t } i} - 2a(m)V_i(m) + \underbrace{V_i^2(m)}_{\text{constant} \leq BE_b} \\ &= \arg \min_i \sum_m -2a(m)V_i(m) = \arg \max_i \sum_{m=1}^{2^{B-1}} a(m)V_i(m) \end{aligned}$$

→ Now $V_i(m)$ is nonzero only for one position m^*

And in that position:

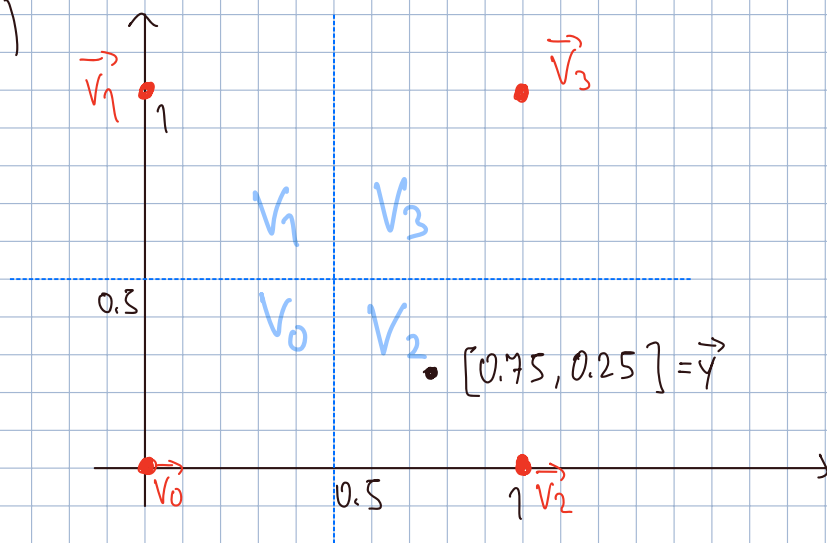
→ if it is positive, then $\hat{y}_i = 2m^*$
→ if its negative, then $\hat{y}_i = 2m^* - 1$

→ As stated before, we find m^* by checking the max. absolute value of $a(m)$

$$m^* = \arg \max_m |a(m)|$$

$$\hat{y}_i = \begin{cases} 2k^* & , \quad a(m) > 0 \\ 2k^* - 1 & , \quad a(m) < 0 \end{cases}$$

3.a)



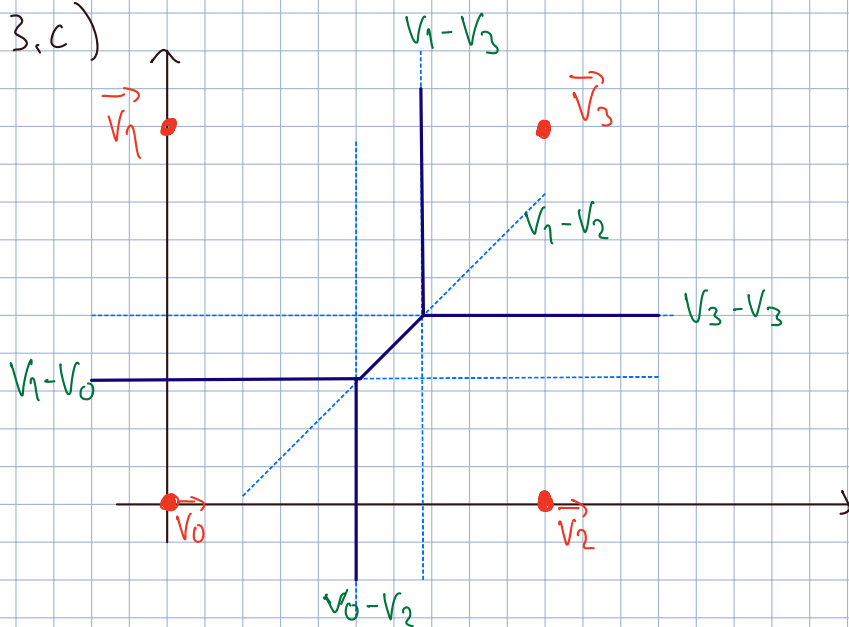
Since all symbols are likely and channel is AWGN, decision rule boils down to nearest neighbour rule.

$\Rightarrow$ We see that $\vec{y}$ is closest to $\vec{V}_2$ and $\vec{b} = [1, 0]$

3.b) Since all symbols are equally likely decision boundaries goes through midpoints of symbols, extending vertically and horizontally from $[0.5, 0.5]$. All points will have same error prob.

All V_k are equally vulnerable under ML rule

3.c)

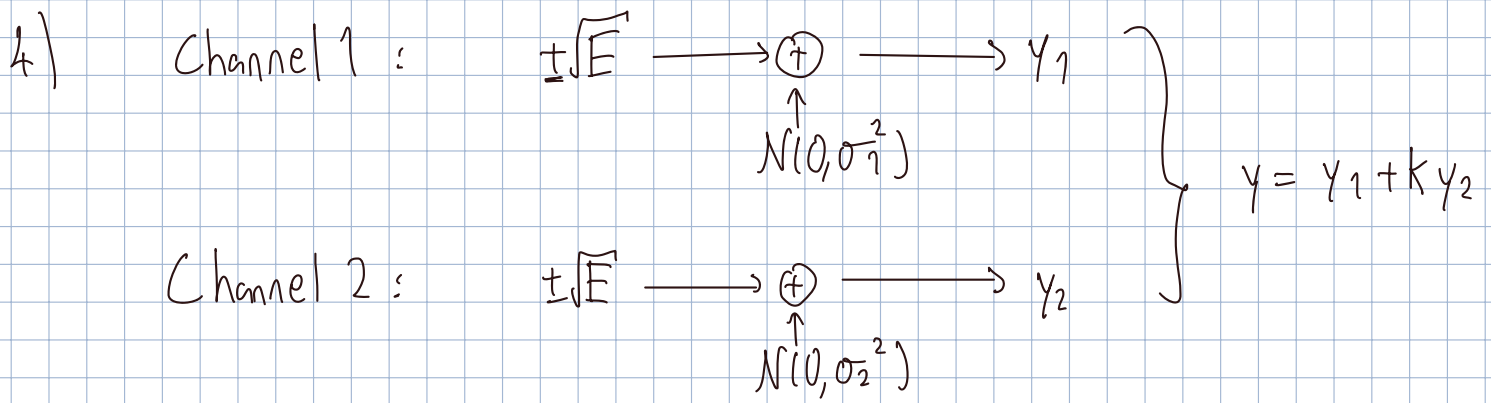


Possible decision boundaries are sketched here

$\hookrightarrow$ Since $\vec{V}_1 = [0, 1]$ is twice as likely, decision boundaries will move further away from $\vec{V}_1$ making its decision region larger.

$\hookrightarrow$ Having a large decision region means there is more room for error

$\rightarrow$ Thus $\vec{V}_1 = [0, 1]$ will be least vulnerable to errors.



4.a) Since $y_1 \sim N(\pm\sqrt{E}, \sigma_1^2)$ and $y_2 \sim N(\pm\sqrt{E}, \sigma_2^2)$ and since sum of Gaussians is also a Gaussian with:

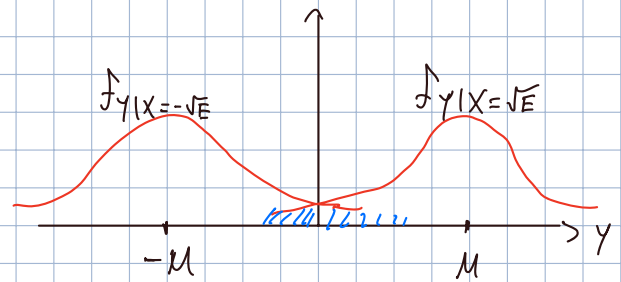
$$\left. \begin{aligned} \mu &= \sqrt{E} + k\sqrt{E} = (1+k)\sqrt{E} \\ \sigma^2 &= \sigma_1^2 + k^2\sigma_2^2 \end{aligned} \right\} y \sim N(\pm\mu, \sigma^2)$$

$$f_{y|x}(y|x=\sqrt{E}) = \frac{1}{\sqrt{2\pi\sigma^2}} e^{-\frac{(y-\mu)^2}{2\sigma^2}}$$

bit 1 sent

$$f_{y|x}(y|x=-\sqrt{E}) = \frac{1}{\sqrt{2\pi\sigma^2}} e^{-\frac{(y+\mu)^2}{2\sigma^2}}$$

bit 0 sent



Assuming equal probs of transmitting bit 0 & bit 1, error prob for ML rule is

$$P_e = 2 \times \frac{1}{2} Q\left(\frac{\mu}{\sigma}\right) = Q\left(\frac{(1+k)\sqrt{E}}{\sqrt{\sigma_1^2 + k^2\sigma_2^2}}\right) = Q\left(\sqrt{\frac{(1+k)^2 E}{\sigma_1^2 + k^2\sigma_2^2}}\right)$$

→ Minimizing error prob. is equivalent to maximizing the argument of Q-function since it is monotonically decreasing.

→ Setting the derivative of term inside square root to 0

$$\frac{d}{dk} \left(\frac{(1+k)^2}{\sigma_1^2 + k^2\sigma_2^2} \right) = \frac{2(1+k)(\sigma_1^2 + k^2\sigma_2^2) - (1+k)^2 2k\sigma_2^2}{(\sigma_1^2 + k^2\sigma_2^2)^2} = 0$$

$$\sigma_1^2 + \sigma_2^2 k^2 = (1+k)k\sigma_2^2 = k\sigma_2^2 + k^2\sigma_2^2 \Rightarrow \sigma_1^2 = k\sigma_2^2$$

$$\hookrightarrow k = \frac{\sigma_1^2}{\sigma_2^2}$$

$$\hat{y} = y_1 + \frac{\sigma_1^2}{\sigma_2^2} y_2^2$$

Optimal estimator

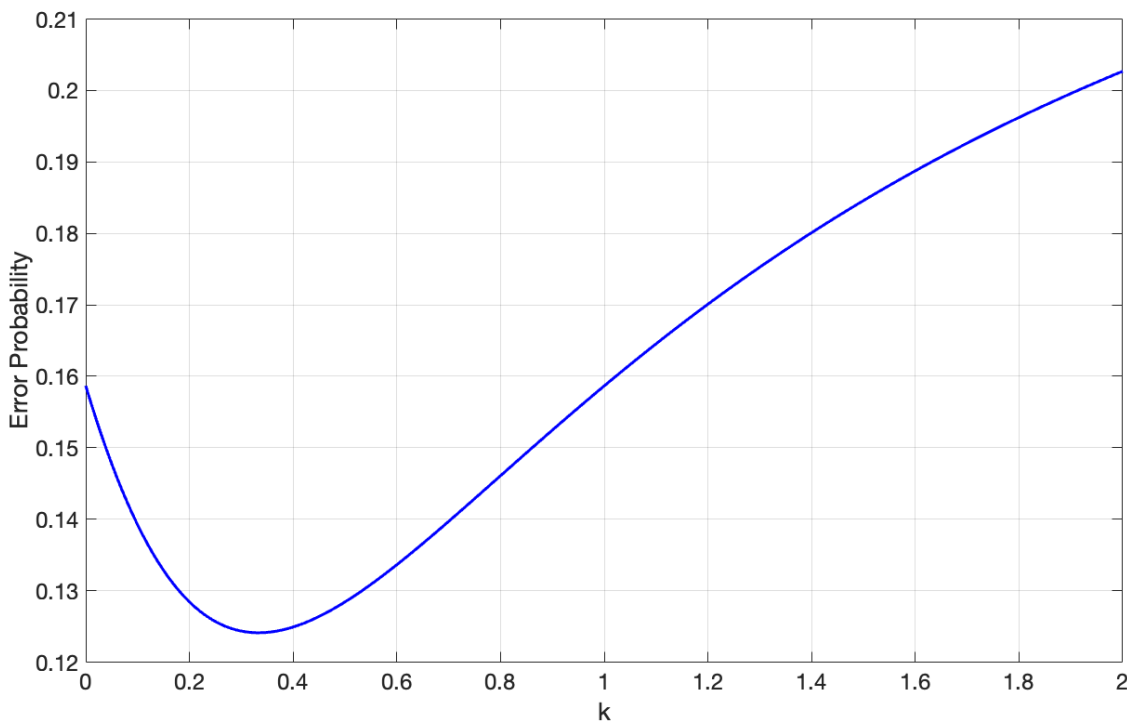
$\Rightarrow$ Makes sense because we want to give more weight to either of y_1 or y_2 with less noise, or penalize the one with more noise.

b)

$$P_e = Q\left(\frac{1+k}{\sqrt{1+3k^2}}\right)$$

```
k = 0:0.01:2;
P_e = qfunc((1+k)./sqrt(1+3.*k.^2));

figure;
plot(k, P_e, 'Color', 'b', 'LineWidth', 3);
xlabel('k', 'FontSize', 24);
ylabel('Error Probability', 'FontSize', 24);
set(gca, 'FontSize', 24);
grid on;
```



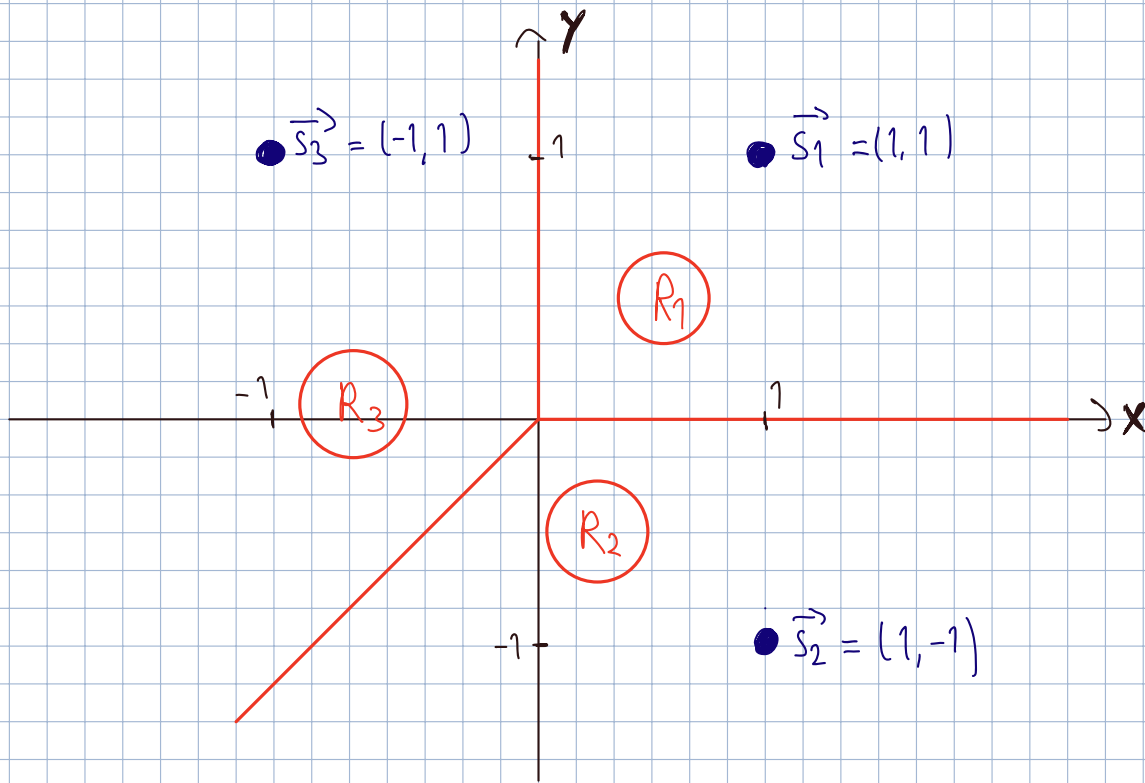
c) $k_{opt} = \frac{\sigma_1^2}{\sigma_2^2} = \frac{1}{3}$. Assume $E = 1$

For $k=1$: $P_e = Q\left(\frac{1+1}{\sqrt{1+3}}\right) = Q(1) = 0.1587$

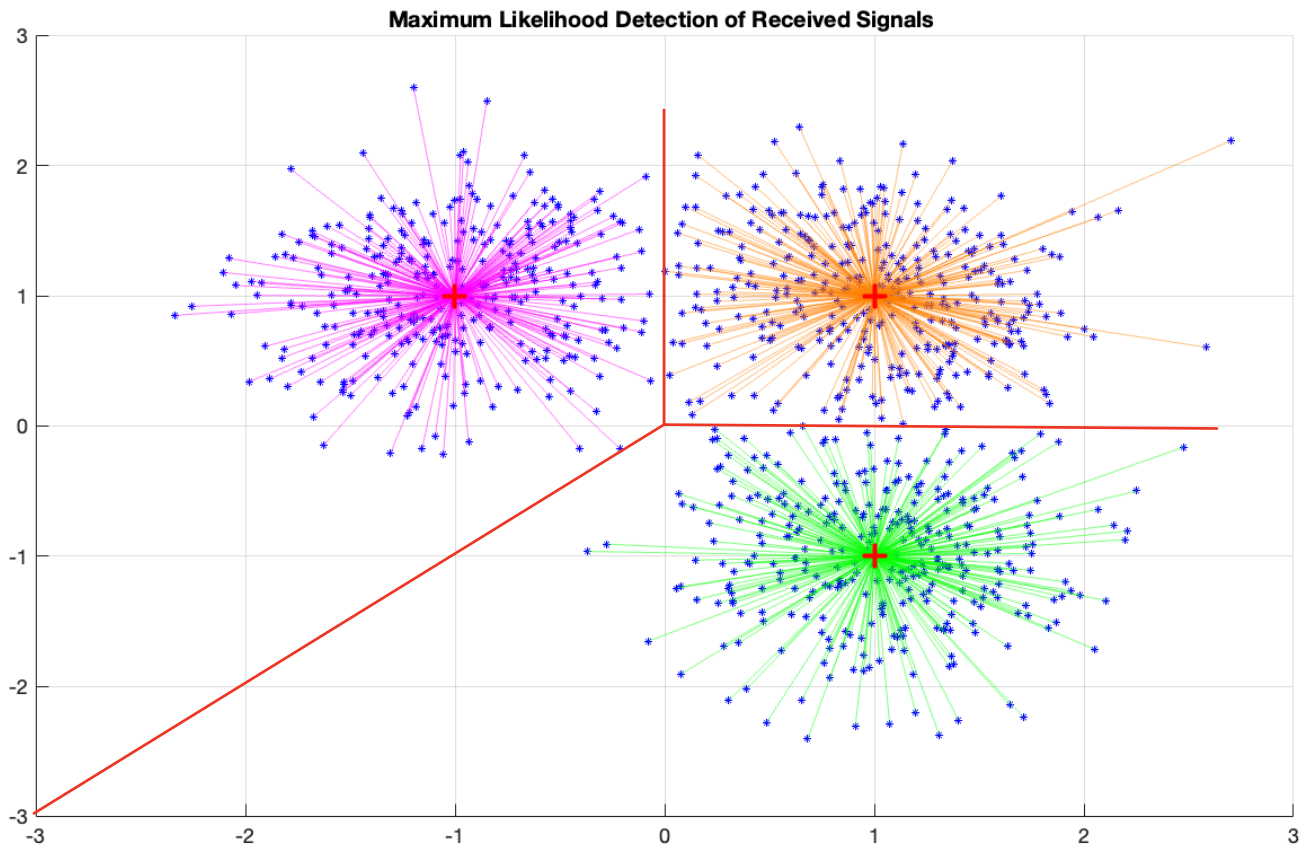
For $k_{opt} = \frac{1}{3}$: $P_e = Q\left(\frac{1+1/3}{\sqrt{1+1/3}}\right) = Q\left(\sqrt{\frac{4}{3}}\right) = 0.1241$

Improved error probability when $k = k_{opt} = 1/3$

5.a)



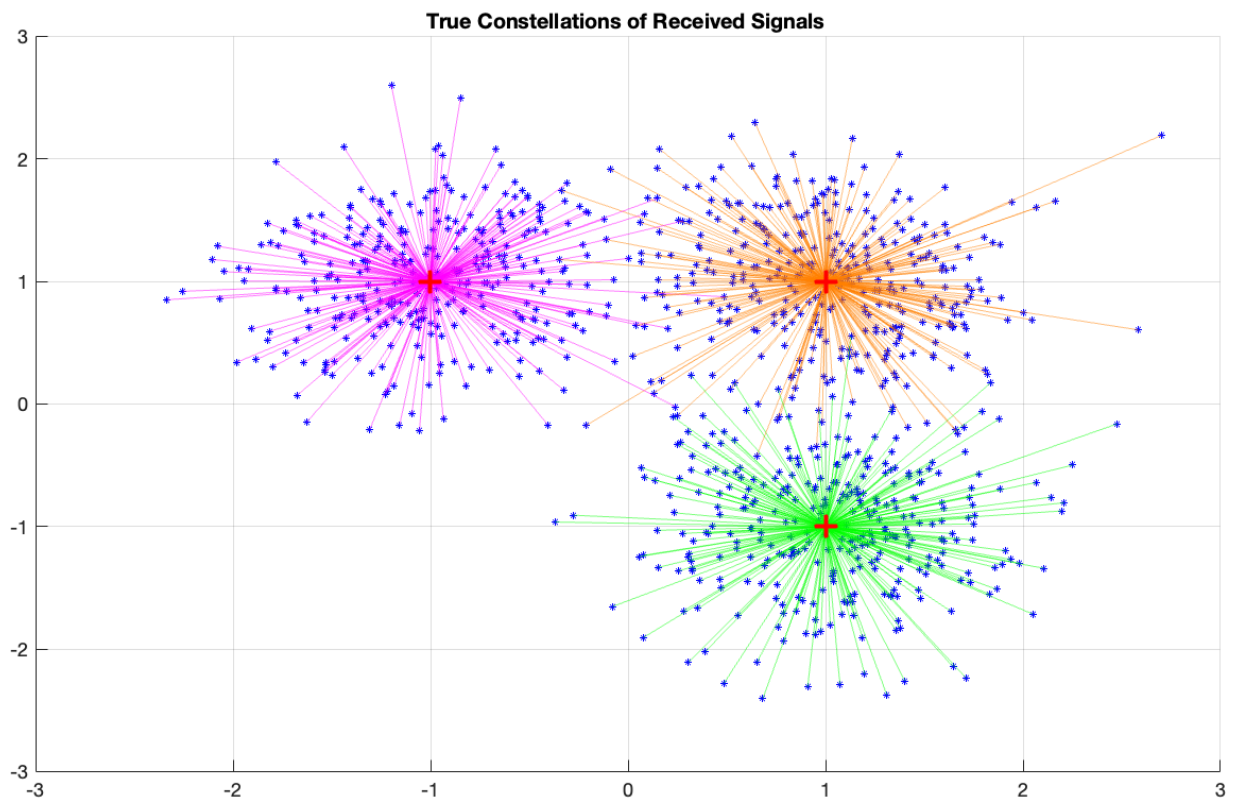
5.b)



→ Decision regions sketched in part a matches perfectly with detected symbols as can be seen above.

→ No line crosses the boundaries (in another words)

5.c)



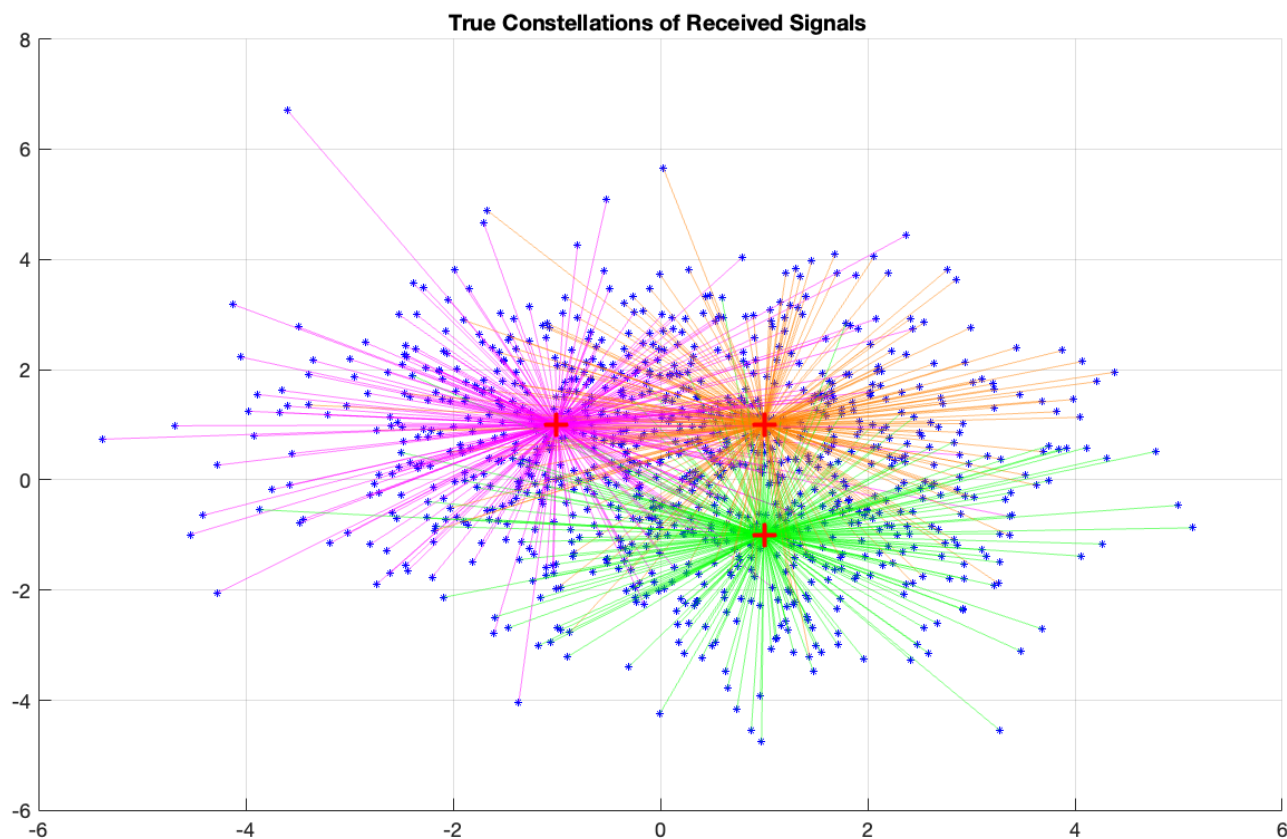
→ We notice that ML rule misclassified some of the signals because some symbols spilled out to the decision region of other symbols due to noise.
We calculate the error prob. as $P_e = 0.032$

3.d)



→ Clearly, there is larger noise in this dataset since transmitted signals are much more dispersed in the constellation space.

→ The boundary between symbols $(1, -1)$ and $(-1, 1)$ is also much more apparent now.



→ As expected, there are much more errors since received signals can be located much further away from original symbols.

→ Error is $P_e = 0.333$ in this case.

5.e)

```
1 data = load("vector_awgn_data2.mat");
2 S = data.S; D= data.D; R=data.R;
3
4 %% Option B
5 figure(1);
6 %Plot received signal vectors y[k]
7 scatter(R(:,1),R(:,2), 'b*', 'LineWidth', 1)
8 hold on;
9
10 %Determine most likely symbols from ML rule
11 repeated_R = repmat(R, [1 1 3]); size(repeated_R)
12 repeated_S = repmat(reshape(S.',[1 2 3]), [1000,1,1]); size(repeated_S)
13 dist = sqrt(sum((repeated_R-repeated_S).^2,2));
14 [min_dist, ML_rule]= min(squeeze(dist),[],2);
15
16 %Plot lines connecting received signals to nearest symbol
17 colors = [[1 0.5 0 0.5] ;[0 1 0 0.5]; [1 0 1 0.5]];
18 for i=1:length(ML_rule)
19     signal = R(i,:);
20     nearest_symbol = S(ML_rule(i),:);
21     plot([signal(1) nearest_symbol(1)], [signal(2) nearest_symbol(2)], 'Color', colors(ML_rule(i),:));
22 end
23
24 %Plot signal vectors s1, s2, s3 (plotted last for visibility purposes)
25 scatter(S(:,1),S(:,2), 'MarkerEdgeColor','r','LineWidth', 48);
26
27 set(gca, 'FontSize',18)
28 title('Maximum Likelihood Detection of Received Signals ')
29 grid on;
30
31
32 %% Option C
33 figure(2);
34 %Plot received signal vectors y[k]
35 scatter(R(:,1),R(:,2), 'b*', 'LineWidth', 1)
36 hold on;
37
38 [~, row_indices] = min(pdist2(D, S), [], 2);
39 %Plot lines connecting received signals to true symbol
40 for i=1:size(D,1)
41     signal = R(i,:);
42     plot([signal(1) D(i,1)], [signal(2) D(i,2)], 'Color', colors(row_indices(i),:));
43 end
44
45 %Plot signal vectors s1, s2, s3 (plotted last for visibility purposes)
46 scatter(S(:,1),S(:,2), 'MarkerEdgeColor','r','LineWidth', 48);
47
48 set(gca, 'FontSize',18)
49 title('True Constellations of Received Signals ')
50 grid on;
51
52 % Calculate Error
53 % Robust Error Calculation
54 P_e = 1 - sum(ML_rule(:) == row_indices(:)) / size(D, 1);
55 disp(P_e)
```